Name: Tyler Coutermarsh
Date: 2/24/2026
Course: Python 100
Assignment: 05

Advanced Collections and Error Handling

Introduction

In this assignment, I will demonstrate how to utilize JSON files in the program instead of csv. As well as utilizing structured error handling with the try-except and finally block.

JSON and Structured Error Handling

I first replace the file name constant with the JSON file that contains the data now instead of the csv.

```
# Define the Data Constants
FILE_NAME: str = "Enrollments.json"

# Define the Data Variables and constants
student_first_name: str = ''  # Holds the first name of a student entered by the user.
student_last_name: str = ''  # Holds the last name of a student entered by the user.
course_name: str = ''  # Holds the name of a course entered by the user.
student_data: dict = {}  # one row of student data (TODO: Change this to a Dictionary)
students: list = []  # a table of student data
csv_data: str = ''  # Holds combined CSV data. Note: Remove later since it is NOT needed with the JSON File
file = None  # Holds a reference to an opened file.
menu_choice: str  # Hold the choice made by the user.
```

Next, I load the current data in the JSON file into the students list using the dump function. This is where the first try except block is utilized. The program tries the code between try and except line by line. If it fails it jumps to the except blocks. In this case if the file is not found and if not that error, then an unknown other error. Each except block displays a custom error message that makes sense to a user. As well as the error message from python.

```
# When the program starts, read the file data into a list of lists (table)
# Extract the data from the file
try:
    file = open(FILE_NAME, "r")
    students = json.load(file)
    file.close()
except FileNotFoundError as e:
    print("Text file must exist before running this script!\n")
    print("-- Technical Error Message -- ")
    print(e, e.__doc__, type(e), sep='\n')
except Exception as e:
    print("There was a non-specific error!\n")
    print("-- Technical Error Message -- ")
    print(e, e.__doc__, type(e), sep='\n')
# Check if a file object exists and is still open
if file is not None and file.closed == False:
    file.close()
```

Lastly, use the dump function to take the new data in the students list to go back into the JSON file along with the original data before the program was run. The difference here is using write mode instead of read mode. Structured error handling is also used but along with the finally block. This block closes the file no matter if there is an error or if the try was run successfully.

```
# Save the data to a file
elif menu_choice == "3":
    try:
        file = open(FILE_NAME, "w")
        json.dump(students, file)
    except FileNotFoundError as e:
        print("Text file must exist before running this script!\n")
        print("-- Technical Error Message -- ")
        print(e, e.__doc__, type(e), sep='\n')
    except Exception as e:
        print("There was a non-specific error!\n")
        print("-- Technical Error Message -- ")
        print(e, e.__doc__, type(e), sep='\n')
    # Check if a file object exists and is still open
    finally:
        if file is not None and file.closed == False:
            file.close()
    print("The following data was saved to file!")
    for student in students:
        print(f"Student {student['FirstName']} {student['LastName']} is enrolled in {student['CourseName']}")
    continue
```

Summary

Structured error handling is great for both letting the user know what is going wrong in English and for letting the developer know what is going wrong from the actual error message to fix the issue and troubleshoot. JSON is the more practical way for transferring data and can be used very similarly to the csv file in the previous assignments.